

## Arduino-Einführung mit I/O

### Worum geht es in diesem Blatt?

In diesem Blatt arbeiten Sie zum ersten Mal mit dem Arduino. Sie schreiben ein erstes Programm, schließen Sensoren und Aktoren an und lassen das Board eigenständig auf Eingaben reagieren. Zum Schluss kommt die drahtlose Kommunikation dazu. Damit haben Sie die Grundlage für alle weiteren Arduino-Blätter dieses Praktikums.

### Was erwarten wir uns von der Praktikumsdokumentation/Protokoll?

- Beantworten der gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Zusätzlich machen sie bitte Screenshots, Fotos, etc. die ihre Schaltungen in „Aktion“ zeigen
- **Eigene Interpretation** von Erkenntnissen des Experiments, Auffälligkeiten
- Welche **Probleme** sind Ihnen begegnet und wie haben Sie diese **gelöst**?

### Wie?

- **Beantworten Sie zur Abgabe den Fragebogen/Quiz „Moodle-Abgabe Blatt 3“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie Ihre Erläuterungen und Quellcode in die jeweiligen Antwortfelder. Screenshots und Bilder sollten Sie anhängen.** (Denn das direkte Einfügen von Bildern in das Moodle-Quiz funktioniert in einigen Browser leider nicht).

### Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

Die Arduino-Plattform “ (<http://www.arduino.cc>) wurde 2005 ins Leben gerufen. Arduino ist eine Open-Source Plattform für Elektronik(-projekte), die eine eigene Entwicklungsumgebung sowie verschieden Input/Output-Boards mit programmierbaren Mikrocontrollern umfasst.

Arduino eignet sich besonders zum schnellen Entwickeln von Prototypen. Der Vorteil liegt in der günstigen Hardware und in der dazugehörigen Entwicklungsumgebung, die die Programmierung und das Übertragen der Programme auf das Arduino-Board stark vereinfacht. Wir verwenden in dieser Übung (bzw. auch in späteren Aufgaben in Kombination mit vvvv) das Board „Arduino UNO R4 WiFi“ mit diversen Sensoren/Aktoren.

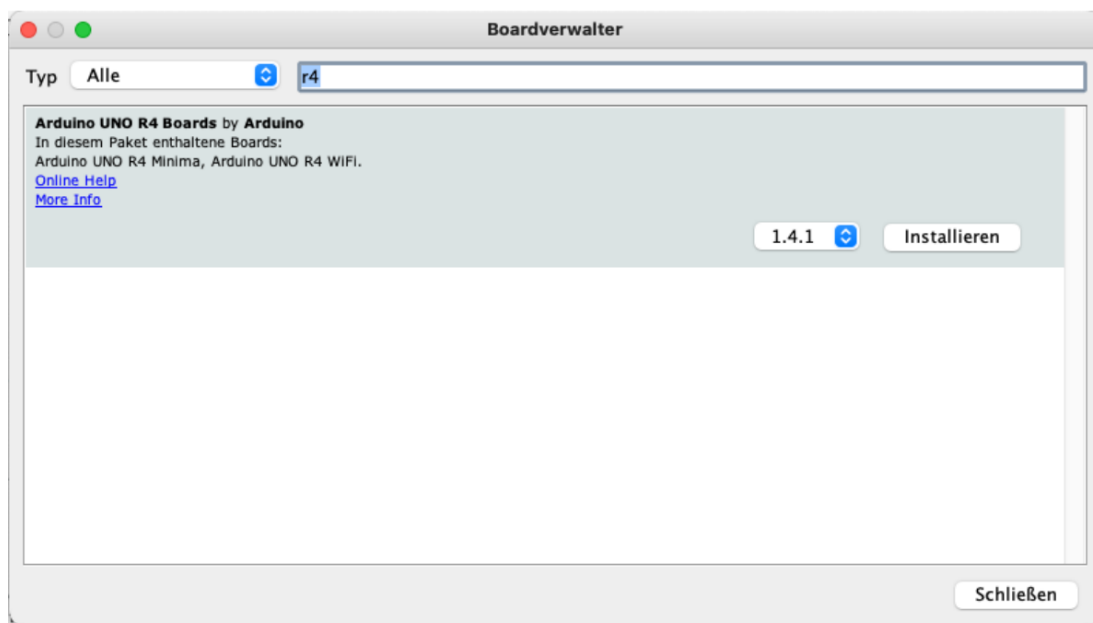
**Bitte beachten Sie zu diesem Aufgabenblatt auch die Powerpoint-Folien „Arduino Mikrocontroller“, die Basisinformationen zu Arduino-Mikrocontrollern, Entwicklungsumgebung und Verwendung, etc. in einer Schritt-für-Schritt Anleitung erläutern.**

## 1) Mein erstes „Arduino-Programm“ (V4)

Im ersten Beispiel geht es darum, das auf dem Arduino Uno V4 Board integrierte Dot-Matrix-Display zu verwenden. Dazu müssen wir zunächst die passende Bibliothek installieren. Auf dem Praktikumsrechner sollte die Bibliothek bereits verfügbar sein, auf Ihrem eigenen Rechner müssen Sie die Bibliothek ggfs. noch installieren.

Installieren und starten Sie die Arduino-Entwicklungsumgebung (<https://www.arduino.cc/en/software/>, auf den Praktikumsrechnern ist diese bereits vorinstalliert). Installieren Sie anschließend die passende Bibliothek für das R4 Board:

- Öffnen Sie dazu zunächst den Boardverwalter: „Werkzeuge“ → „Board: Arduino XXX“ → „Boardverwalter...“
- Geben Sie in dem Suchfeld das Stichwort „R4“ ein und klicken Sie dann auf „Installieren“. Die Installation nimmt üblicherweise ein paar Minuten in Anspruch. Bereiten Sie in der Zwischenzeit schon einmal die für dieses Aufgabenblatt benötigten Komponenten (s.u.) vor.



### Wir benötigen folgende Komponenten

- Arduino UNO-R4 Board
- „Breadboard“
- Externe LED, rot (mit 2 Pins)
- Steckkabel (ca. 5 Stück)

### Vorbereitung

- Am PC die Arduino-Entwicklungsumgebung starten
- Mittels USB-Kabel das Arduino-Board mit dem PC verbinden
- Die Arduino IDE für den korrekten Chip konfigurieren (µC auswählen): Menü „Tools“ → „Microcontroller (MCU)“ → „Arduino Uno R4 Wifi“ auswählen

- Die serielle Schnittstelle konfigurieren: Menü „Tools“ → „Serial Port“ → „COMX“ (richtigen Port auswählen, falls nicht bekannt: siehe Windows-Systemsteuerung)
- Über die Option „File“ → „New“ einen sogenannten neuen Sketch anlegen

### 1.a) Ansteuerung der internen LED

Als allererstes bringen wir die on-board LED des Arduino-Boards zum Blinken. Dieser erste Einstieg erlaubt es auch, das Board auf seine Funktionalität zu prüfen sowie die Korrektheit der Installation zu testen. Die LED soll dabei ein SOS-Signal senden. Das bedeutet:

- Die LED blinkt dreimal kurz („S“), dreimal lang („O“) und wieder dreimal kurz („S“).
- Nach dem Senden des SOS-Signals erfolgt eine Pause von drei Sekunden.
- Im Anschluss wird das SOS-Signal erneut gesendet (Endlosschleife).
- Die LED leuchtet bei einem kurzen Blinken für 0,2 Sekunden und bei einem langen Blinken für 0,6 Sekunden.
- Die Pausen innerhalb des Buchstabens (in welchen die LED aus ist), betragen ebenfalls 0,2 Sekunden und die Pause zwischen den Buchstaben beträgt 0,6 Sekunden.

#### Folgende Informationen sind bekannt:

- Jedes Arduino-Programm enthält eine Setup-Methode (`void setup()`) zu Initialisierungszwecken (hier wird auch die Konfiguration bzw. Zuordnung der Pins zur Ein- oder Ausgabe vorgenommen)
- Initialisierung erfolgt über das Kommando `pinMode(Pinnummer, Modus)` – Pinnummer: siehe Beschriftung direkt am Arduino-Board; Modus: INPUT PIN wird gelesen, OUTPUT an den PIN wird etwas ausgegeben
- sowie eine `loop()`-Routine, die bis zum Programmende immer wieder durchlaufen wird (d.h. am Ende der `loop()`-Schleife wird wieder an dessen Anfang gesprungen)
- Hier werden Eingaben vom Arduino-Board an das Programm übertragen und auch Ausgaben an das Board gemacht
- Das Kommando `digitalWrite(Pinnummer, wert)` gibt den entsprechenden Wert an den Pin Pinnummer aus. Zulässige Werte: HIGH (1, Ein), LOW (0, Aus)
- Mit dem Befehl `delay(Dauer)` wird die Programmausführung für Dauer Millisekunden gestoppt
- Beim Arduino UNO-Board wird die interne LED über Port 13 angesprochen
- **Wichtiger Hinweis:** In der Arduino IDE können Sie über das Kontextmenü bei markiertem Befehl das Hilfesystem aufrufen („In der Referenz suchen“) und detaillierte Informationen zur Verwendung dieses Befehls abrufen

Versuchen Sie nun ihr erstes Programm mit diesen Informationen zu erstellen. Laden Sie dazu das Programm „aufgabe\_1a\_vorlage.ino“ von Moodle (direkt die komplette zip-Datei **laden und entpacken**; den entpackten Ordner nicht unter „Desktop“ oder „Downloads“ ablegen) und ergänzen Sie es um die fehlenden Funktionen. Bevor das fertige Programm auf das Arduino-Board gespielt werden kann, ist dessen Funktion zu überprüfen und das Programm zu kompilieren. Wählen Sie dazu in der Toolbar den zweiten Button („Hochladen“).

**Frage 1:** Funktioniert Ihr Programm? Was macht es?

Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)

**Frage 2:** Wie muss das Programm verändert werden, damit ein SOS-Signal gesendet wird? Geben Sie nachfolgend zusätzlich auch Ihren Programmcode ab.

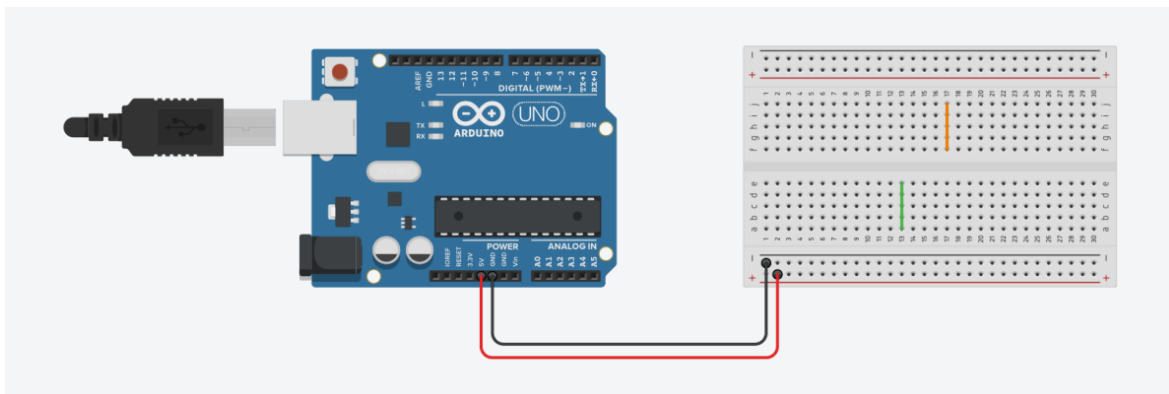
Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)

### 1.b) Ansteuerung einer externen LED

Nehmen Sie nun die rote LED (mit 2 Pins). Wir möchten nun diese LED an einem anderen Pin, z. B. Pin 9 blinken lassen. Dazu müssen wir das Programm anpassen. Wir stecken zunächst das Arduino-Board vom USB-Stecker ab und verwenden das „Breadboard“ um eine Schaltung aufzubauen.

Einschub: Was ist das „Breadboard“ (=Steckplatine, Steckbrett) und wie bauen wir damit Schaltungen auf? Ein „Breadboard“ verwenden wir zur elektrischen Verbindung von elektronischen Bauteilen für Versuchsschaltungen und Experimente.

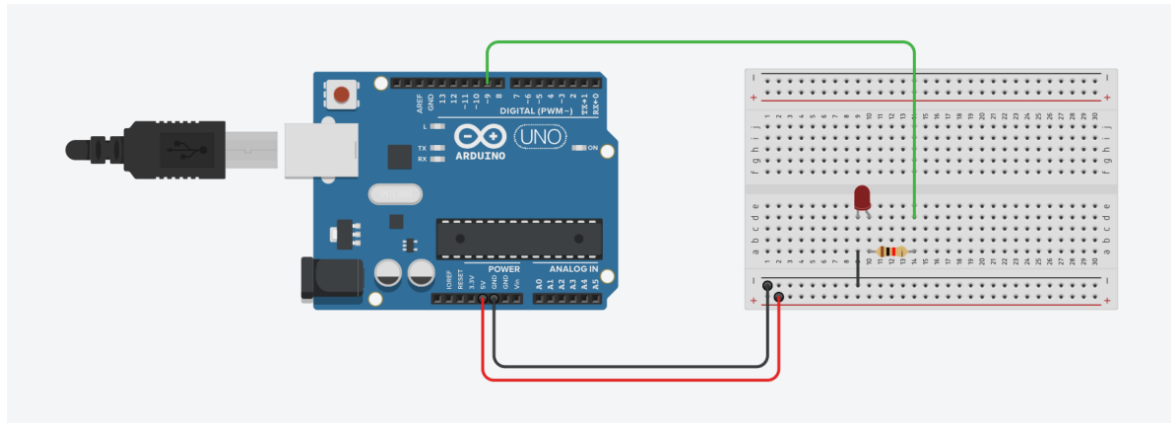
- Sämtliche Pins in den beiden äußeren Reihen oben und unten sind horizontal miteinander verbunden. Dort legt man in der Regel Versorgungsspannung (+5 V) und Masse (GND) an, und kann es von dort einfach in die vertikalen Pinreihen verteilen
- Die vertikalen Reihen im Mittelteil sind ebenfalls jeweils intern miteinander verbunden, und zwar A-E und F-I; in der Abbildung unten sind beispielsweise die grün bzw. gelb markierten Pins miteinander verbunden



Folgende Schaltung wollen wir nun aufbauen. Wie in der Abbildung zu sehen, brauchen wir neben der LED noch ein zweites Bauteil – einen sogenannten „Vorwiderstand“. Bevor wir also die Schaltung aufbauen können, müssen wir diesen Vorwiderstand zunächst ausrechnen.

Zur Berechnung können Sie im Internet einen Widerstandsrechner finden, z.B. auf <https://www.leds-and-more.de/catalog/resistor.php/>

Hinweise zur Berechnung: Durchlassspannung der LED: 1,5 Volt, Betriebsstrom: 8 mA, das Arduino-Board liefert 5,0 Volt Betriebs- bzw. Versorgungsspannung



**Frage 3:** Wozu brauchen wir diesen Vorwiderstand? Wie groß muss der sein (geben Sie die Berechnung an)?

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle)*

Bauen Sie nun die Schaltung mit dem korrekten Vorwiderstand auf. (Bitte versichern Sie sich bei den Praktikumsbetreuern, dass Ihre Berechnung korrekt ist – damit die LED nicht zerstört wird).

**Frage 4:** Welche Änderungen müssen Sie am Arduino-Programm vornehmen, damit die rote externe LED nun (an Pin 9) blinkt?

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

### 1.c) Dimmen von LEDs (mit Pulsweitenmodulation)

Eines gleich vorweg: Eine LED lässt sich im Gegensatz zu einer herkömmlichen Glühlampe auf „normale Weise“ nicht dimmen, sondern lediglich ein- oder ausschalten. Wird eine LED jedoch sehr schnell abwechselnd ein- und wieder ausgeschaltet (bezeichnet man als „Pulsung“), so nehmen wir Menschen eine geringere Helligkeit wahr; damit kann also eine „Dimmung“ erreicht werden.

**Folgende Informationen sind bekannt:**

- Jedes Arduino-Board besitzt Ports die für Pulsweitenmodulation (PWM) genutzt werden können. Beim Arduino UNO sind das die Pins 3, 5, 6, 9, 10, 11
- PWM erzeugt man mittels des Kommandos `analogWrite(Pinnummer, wert)`. Achtung: funktioniert nur an einem PWM-Port (sind gekennzeichnet mit „~“)
- Pinnummer: siehe Beschriftung direkt am Arduino-Board (jene mit „~“)
- wert: Gibt den sogenannten „duty cycle“ an und liegt zwischen 0 (immer aus) und 255 ( $2^8-1$ , immer ein)

- Am Ausgang wird nun ein kontinuierliches Rechtecksignal mit entsprechendem Puls-Pausenverhältnis erzeugt (Frequenz: 490/980 Hz)

Versuchen Sie nun die LED an Pin 9 pulsweitenmoduliert mit verschiedenen Helligkeitswerten (z. B. 50, 100, 200) leuchten zu lassen – verändern Sie das vorherige Programm entsprechend. Vergessen Sie nicht, nach jeder Änderung das Programm zu kompilieren und auf das Arduino-Board zu übertragen.

**Frage 5:** Funktioniert Ihr Programm? Was stellen Sie fest? Wie funktioniert Pulsweitenmodulation (PWM) - können Sie das Skizzieren? Geben Sie nachfolgend auch Ihren Programmcode ab.

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

Damit das ganze etwas dynamischer wird, kann man die LED auch „pulsen“ lassen, d.h. die Helligkeit von 0 (=aus) langsam bis zur vollen Helligkeit ansteigen lassen und dann wieder langsam abnehmen lassen. Das Arduino-Programm „aufgabe\_1c\_vorlage.ino“ können Sie wieder aus Moodle laden.

Ergebnis: Eindeutig ein „schöneres“ Blinken – finden Sie nicht auch? Experimentieren Sie nun mit den Werten für `fadeAmount` und `delay` (vergessen Sie nicht, nach jeder Änderung das Programm zu kompilieren und auf das Arduino-Board zu spielen).

**Frage 6:** Wie funktioniert das Programm? Was beobachten Sie?

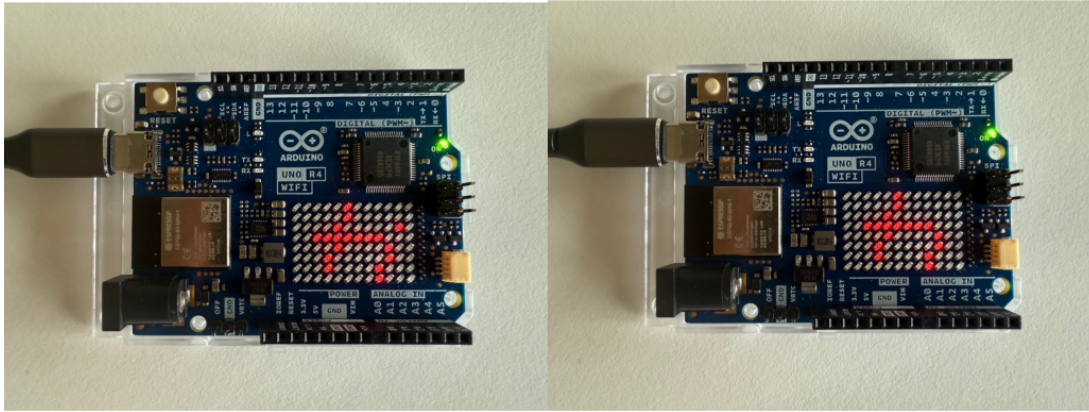
*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

### 1.d) Ansteuerung des internen Displays

Installieren Sie anschließend die Bibliothek für das LED Matrix Display. Gehen Sie dazu wie folgt vor:

- Öffnen Sie dazu zunächst die Bibliotheksverwaltung: „Werkzeuge“ → „Bibliotheken verwalten“
- Geben Sie in dem Suchfeld das Stichwort „HT16K33“ ein und klicken Sie dann auf „Installieren“.

Wir wollen nun auf dem internen Display das THI-Logo darstellen. Das Breadboard ist für diese Aufgabe nicht erforderlich – Sie können das also vom Arduino abstecken (lassen Sie aber die Bauteile auf dem Breadboard stecken – wir brauchen das Board in Aufgabe 2 noch). Das Programm wird am Ende dieser Aufgabe dann so aussehen:



Auf dem Display wird also das THI-Logo dargestellt – und der Punkt auf dem „i“-Zeichen soll dabei regelmäßig blinken. Öffnen Sie für diese Aufgabe zunächst das Arduino-Programm „aufgabe\_1d\_vorlage.ino“. Das Programm lässt sich bereits ausführen.

**Frage 7:** Wie funktioniert das Programm? Was beobachten Sie?

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

Versuchen Sie zu verstehen, wie die einzelnen Frames aufgebaut sind. Ein Frame wird dabei in dem Arduino-Programm wie folgt codiert:

```
{
  0x80000000,
  0x00000000,
  0x00000000
}
```

Tipp: Verschieben Sie die Zahl „8“ Stück für Stück nach rechts – sowie in die Zeilen 2 und 3. Verwenden Sie statt der Zahl 8 testweise auch die Zahlen 1, 2 und 4. Was beobachten Sie? Was passiert, wenn Sie andere (hexadezimale) Zahlen zwischen 0 und f verwenden?

**Frage 8:** Definieren Sie in Ihrem Programm nun zwei Frames, so dass die ersten vier LEDs und die letzten vier LEDs abwechselnd blinken.

*Dokumentieren Sie den Programmcode in ihrem Protokoll (auf Moodle)*

Nun wollen wir die zwei Varianten des THI-Logos erstellen. Versuchen Sie zunächst, dazu die einzelnen LEDs über das Array anzusteuern. Falls Ihnen das mittels Ihres Programmcodes nicht gelingt, können Sie auch den LED Matrix Editor (<https://ledmatrix-editor.arduino.cc>) als kleine Hilfestellung verwenden. Dort wird ein Frame standardmäßig in vier Zeilen codiert. Die vierte Zeile benötigen wir in unserem Beispiel nicht – diese können Sie entfernen (die Zeile hat aber auch keinen negativen Effekt auf die Darstellung – maßgeblich sind nur die Zeilen 1 bis 3 Ihres Frames).

**Frage 9:** Erstellen Sie ein Programm, das ein animiertes THI-Logo darstellt.

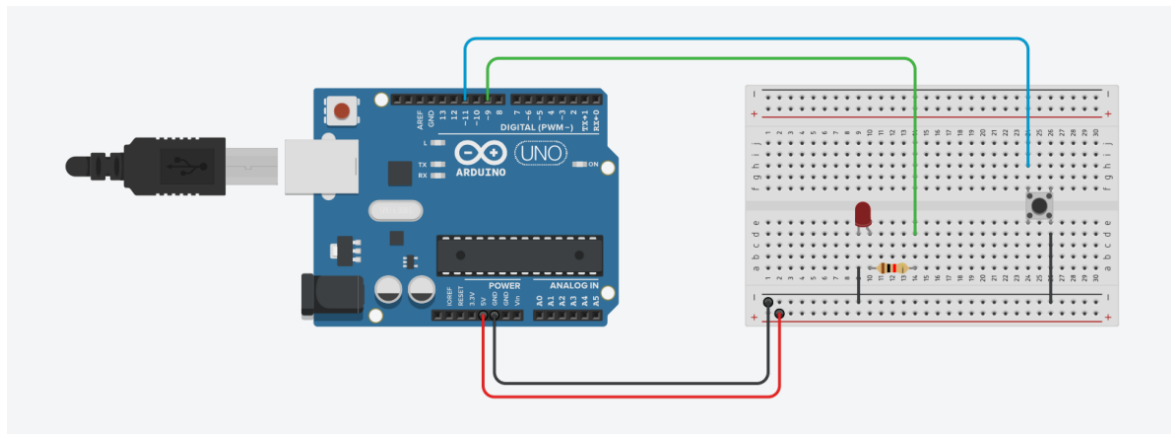
*Dokumentieren Sie den Programmcode in ihrem Protokoll (auf Moodle)*

## 2) Arduino als intelligenter Controller

In diesem Beispiel lernen Sie nun, dass man das Arduino Board als Standalone-Controller für Ein- und Ausgabe betreiben kann. Mit externer Spannungsversorgung läuft die komplette Logik auf dem Board, USB-Kabel bzw. der Computer würde dazu nicht mehr notwendig sein – wir brauchen Computer und USB-Kabel also nur noch zur Spannungsversorgung.

### 2.a) Button steuert LED

In der nachfolgenden Schaltung und dem dazugehörigen Programm verwenden wir einen „Push-Button“, um direkt die LED zu steuern. Wir stecken das Arduino-Board wieder vom USB-Anschluss ab und erweitern die Schaltung auf dem „Breadboard“ wie nachfolgend dargestellt um dieses Verhalten zu implementieren.



**Noch ein Hinweis zu den Drahtfarben in der Schaltung:** „Schwarz“ kennzeichnet in der Regel „Masse“ (0 Volt, GND), „rot“ steht für Versorgungsspannung (hier: +5 Volt), weitere beliebige Farben werden zur Signalübertragung von Sensoren, etc. verwendet

Ansonsten sieht der Aufbau nun wie folgt aus:

- Wann immer mehrere Komponenten auf Masse oder Versorgungsspannung zugreifen, legt man diese beiden Signale auf die äußeren Reihen des „Breadboards“ – alle Pins der 4 äußeren Reihen sind (pro Reihe) miteinander verbunden.
- Die LED bleibt über den Vorwiderstand (Größe wie zuvor) an Pin 9 angeschlossen.
- Der „Push-Button“ ist wie auf dem Schaltplan gezeigt anzuschließen. Die beiden linken Pins sind fest verdrahtet (ob sie das blaue Kabel links unten oder links oben anschließen, ist also egal) – das gleiche gilt für die beiden rechten Pins.
- Wenn der Button nicht gedrückt ist, besteht jedoch keine Verbindung zwischen der linken und der rechten Seite des Buttons (also zwischen dem blauen und schwarzen Kabel). Wenn der Button gedrückt ist, wird eine Verbindung zwischen diesen beiden Anschlüssen hergestellt (der PIN 11 wird also „auf Masse gezogen“).

Öffnen Sie nun das Programm `aufgabe_2a_vorlage.ino`. Zu Beginn werden die Variablen `buttonPin` und `ledPin` definiert. Wenn Sie die LED bzw. den Button an anderen Pins anschließen möchten, dann müssen Sie diese Variablen entsprechend anpassen.

In der `setup()`-Methode legen wir die Zuordnungen fest (Button = Eingabe, LED = Ausgabe), d.h. an einen Pin werden Signale empfangen, am anderen Pin werden Signale gesendet. Wenn der Button offen ist, ist er weder mit GND noch mit VCC verbunden. Damit „floatet“ er – liefert also zufällige Werte, wenn wir ihn als „normalen“ INPUT definieren würden. Wenn wir als `pinMode` jedoch `INPUT_PULLUP` verwenden, wird ein interner, sogenannter PullUp-Widerstand verwendet. Das bedeutet: Wenn der Button offen ist, dann liefert uns der Arduino an diesem Pin konstant einen HIGH-Pegel.

In der `loop()`-Routine wollen wir nun wie folgt vorgehen: Wir lesen zunächst den Status des Buttons ein, speichern den Wert in der Variablen „`buttonState`“ und steuern damit, abhängig vom Wert (LOW bzw. HIGH) die LED an. Wichtig ist, dass das Verhalten genau invertiert ist: Wenn der Button gedrückt wird, ist er LOW und die LED soll leuchten (HIGH). HIGH und LOW sind Konstanten, können also (ohne Anführungszeichen) in Ihrem Code verwendet werden.

Der „Push-Button“ ist an Pin 11, einem digitalen Pin, angeschlossen. Zum Einlesen des Status verwenden wir die Methode `erg = digitalRead(pin)` (und setzen dort die korrekten Variablennamen für `erg` und `pin` ein. Abhängig vom ermittelten Wert wird die LED ein- bzw. ausgeschaltet. Dazu verwenden wir wiederum die Funktion `digitalWrite(pin, wert)`. Sie können entweder mit einer Fallunterscheidung (`if / else`) arbeiten – oder die Negation (`!wert`) verwenden.

Versuchen Sie die Schaltung und das Programm fertigzustellen, kompilieren Sie Ihre Lösung und laden Sie sie auf das Arduino Board.

**Frage 10:** Funktioniert die Interaktion wie gewünscht? Wie sieht Ihr finaler Code aus – kopieren Sie wiederum Ihre Lösung in das Protokoll bzw. Upload. Gerne können Sie auch noch ein Foto Ihrer Schaltung (Arduino+Breadboard+Verkabelung) machen und mitabgeben.

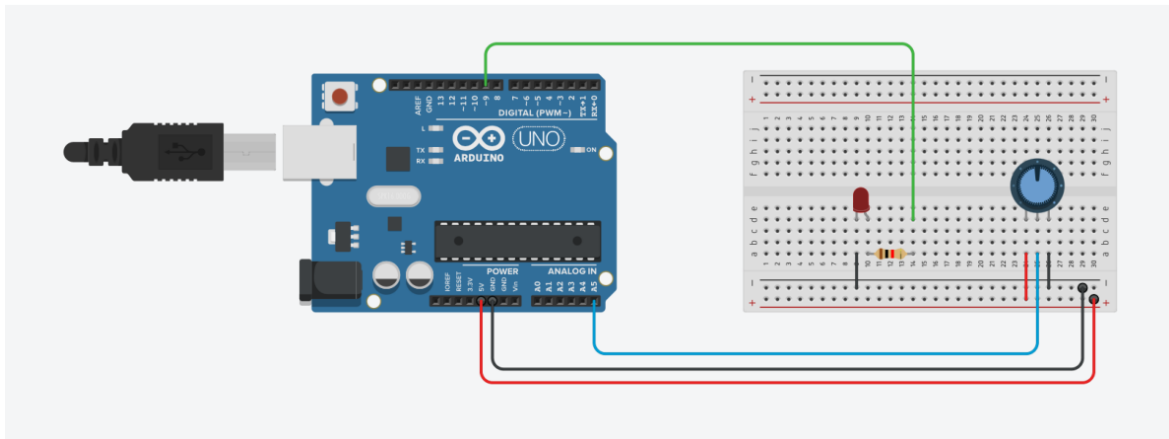
*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

## 2.b) Potentiometer steuert LED-Helligkeit

Als nächstes möchten wir mit einem Drehregler („Potentiometer“) direkt die Helligkeit einer LED linear verändern. Vorgehensweise:

Wir stecken das Arduino-Board wieder vom USB-Anschluss ab und modifizieren die Schaltung von zuerst (auf dem „Breadboard“) so wie nachfolgend dargestellt. Die Potentiometer haben wir dabei in zwei Ausführungen. In der abgebildeten, schwarzen Variante – und als etwas kompakteres Bauteil („blau“). Die blauen Bauteile funktionieren dabei etwas robuster.

**Der Drehregler funktioniert wie folgt:** Der linke Pin ist an die Versorgungsspannung (VCC, +5 V) zu legen, der rechte Pin an Masse (GND, 0 V). Über den mittleren Pin wird die Spannung des Spannungsteilers – entsprechend der Stellung des Potentiometers – abgegriffen (kontinuierlicher Wert von 0 bis +5 Volt). Dieser analoge Wert kann nun über einen Analogeingang am Arduino-Board im Programm weiterverwendet werden (A0 bis A5). Die LED bleibt weiterhin über den Vorwiderstand (Größe wie zuvor) an Pin 9 angeschlossen



Öffnen Sie nun das Programm „aufgabe\_2b\_vorlage.ino“ zum Abbilden der Logik. Wir verwenden auch hier wieder einige globale Variablen (noch vor der `setup()`-Methode): Eine Variable zur Definition des Eingabepins für den Drehregler. Dieser soll über den Anschluss A5 angeschlossen werden. Eine Variable zur Definition des Ausgabepins für die LED. Soll wie zuvor Pin 9 sein.

In der `setup()`-Methode legen wir die Zuordnungen fest (Drehregler = Eingabe, LED = Ausgabe), d.h. an Pin A5 sollen Signale empfangen werden, an Pin 9 Signale gesendet werden. In der `loop()`-Routine lesen wir zunächst den Wert des Potentiometers ein und speichern diesen Wert in der zuvor definierten Variablen für den Wert des Drehreglers. Um die LED ansteuern zu können, ist zuvor eine Umrechnung notwendig:

Der Wert des Drehreglers wird über die Funktion `erg = analogRead(pin)` eingelesen (`erg`, `pin` sind Platzhalter für „unsere“ Variablen). Das Board kann nicht direkt mit analogen Spannungen arbeiten, sondern „digitalisiert“ diese Werte mittels eines sogenannten Analog-Digital-Konverters (ADC). Beim Arduino UNO R4 handelt es sich um einen 14-Bit-ADC, d. h. der Eingangsbereich von 0 bis 5 Volt wird auf Integer-Werte von 0 bis 16.383 ( $2^{14} = 16.384$  Stufen) abgebildet. Die Variable kann also Werte im Bereich 0 bis 16.383 annehmen.

Wie wir aus dem vorherigen Beispiel zur Pulsweitenmodulation wissen, kann die Funktion `analogWrite(Pinnummer, Wert)` verwendet werden, um eine LED am Ausgang zu „dimmen“. Der Wertebereich für `Wert` liegt dabei weiterhin zwischen 0 und 255 (= 8 Bit). Um den Eingabewert des Potentiometers direkt zum Dimmen der LED verwenden zu können, muss der Wertebereich 0 bis 16.383 (des ADC) auf 0 bis 255 (für PWM) umgerechnet werden. Dies geschieht mit der Funktion `map()`, z. B. `erg = map(eingabe, 0, 16383, 0, 255)`.

Versuchen Sie die Schaltung fertigzustellen, kompilieren Sie Ihre Lösung und laden Sie sie auf das Arduino Board. Der Drehregler hält unter Umständen nicht besonders gut am „Breadboard“ – zur Abhilfe können Sie gerne 30cm „Male-Female Extenderkabel“ aus dem „Prototyping Interfaces Starterkit“ benutzen.

**Frage 11:** Funktioniert die Interaktion wie gewünscht? Gerne können Sie auch noch ein Foto Ihrer Schaltung (Arduino+Breadboard+Verkabelung) machen und mit Ihrer Lösung mitabgeben.

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

**Eine Möglichkeit für „Debugging“:** Nachdem wir am Arduino-Board kein großes Display zur Verfügung haben, um alle Werte ausgeben zu können, müssen wir eine andere Möglichkeit finden, um unser Programm debuggen zu können. Insbesondere interessieren wir uns häufig für den Inhalt bestimmter Variablen. Variableninhalte können wir auf die serielle Schnittstelle ausgeben und über den sogenannten „Serial Monitor“ überwachen. Damit haben wir eine einfache Möglichkeit für „Debugging“ - und so geht's:

In der `setup()`-Methode fügen wir folgende Zeile ein, um eine serielle Verbindung zu öffnen: `Serial.begin(115200);` und in der `loop()`-Methode verwenden wir `println` (oder `print`), um Inhalte auf der seriellen Schnittstelle auszugeben: `Serial.println(„Text“);`

Über das Menü „Tools“ → „Serial Monitor“ können wir nun den Inhalt der Variablen überwachen während das Programm auf dem Arduino ausgeführt wird (und der Arduino per USB-Kabel mit dem PC verbunden ist) – sobald wir dort die gleiche Baud-Rate (115200) eingestellt haben.

**Frage 12:** Erweitern Sie ihr Programm um diese Monitoring-Funktion und fügen Sie den geänderten Code sowie einen Screenshot des Serial Monitors hier ein.

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll*

### 3) Drahtlose Kommunikation mit dem Arduino

In diesem Beispiel lernen Sie, wie sich ein Arduino UNO R4 WiFi ohne zusätzliches WLAN mit einem Computer verbinden kann – und zwar über eine selbst aufgebaute WLAN-Verbindung. Der Arduino agiert dabei als Access Point und stellt einem verbundenen Gerät (z. B. einem Java-Programm auf einem PC) eine direkte TCP/IP-Verbindung zur Verfügung.

Der Vorteil: Sie benötigen kein zusätzliches Netzwerk (z. B. keinen Router), sondern können sofort eine direkte Kommunikation aufbauen – etwa zur Steuerung, Fernabfrage oder Visualisierung von Daten.

#### 3.a) Arduino als Access Point und TCP-Server

Im ersten Schritt programmieren wir den Arduino so, dass er ein eigenes WLAN-Netz bereitstellt. Dieses Netz benennen Sie beispielsweise nach Ihren beiden Nachnamen (z.B. „RiemerNestler“) und sichern es mit einem Passwort (mit mindestens **acht Zeichen**). Zusätzlich lauscht der Arduino auf einem bestimmten Port (in unserem Fall: Port 8888) und nimmt dort eingehende Verbindungen von Clients entgegen.

Öffnen Sie nun das Programm `aufgabe_3a_arduino_vorlage.ino`. Zu Beginn sind die SSID (Netzwerkname) und das Passwort für das WLAN festzulegen.

Sobald der Access Point aktiv ist, wird eine feste IP-Adresse vergeben (standardmäßig 192.168.4.1). Über diese IP kann ein verbundenes Gerät nun mit dem Arduino kommunizieren. Im Sketch wird dabei ein Server auf Port 8888 gestartet. In der `loop()`-Methode wartet der Arduino auf eingehende TCP-Verbindungen. Wenn ein Client sich verbindet und Daten schickt, werden diese per `readStringUntil('\n')` empfangen und über den Serial Monitor ausgegeben.

**Frage 13:** Verbinden Sie sich nun mit dem Smartphone und/oder PC mit dem von Ihnen soeben erzeugten WLAN – und erstellen Sie einen Screenshot von der erfolgreich hergestellten Verbindung.

*Dokumentieren Sie Ihre Erkenntnisse in ihrem Protokoll (auf Moodle).*

#### 3.b) Java-Client zur Kommunikation mit dem Arduino

Als Gegenstück zum Arduino benötigen wir eine Client-Anwendung, die über WLAN mit dem Access Point des Arduino verbunden ist und Daten senden kann. Dies realisieren wir mit einem einfachen Java-Programm mit grafischer Benutzeroberfläche.

Öffnen Sie das Projekt `aufgabe_3b_java_vorlage`. Dieses Programm verbindet sich über einen TCP-Socket zur IP-Adresse 192.168.4.1 und Port 8888 (diese Werte sind fest im Programm hinterlegt). Sie können nach dem Start über ein Eingabefeld ein Zeichen oder einen Text eingeben und mit einem Klick auf „Senden“ an den Arduino übertragen.

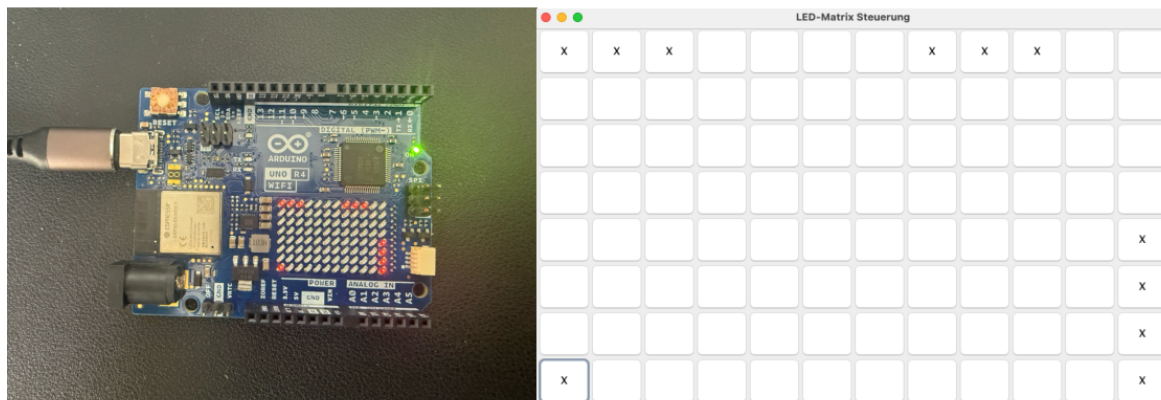
Die Verbindung wird bei jedem Klick neu aufgebaut und nach dem Senden sofort wieder geschlossen. Auf Arduino-Seite wird der empfangene Text zeilenweise eingelesen (`readStringUntil('\n')`) und über den Serial Monitor angezeigt.

**Frage 14:** Funktioniert die Kommunikation zwischen dem Java-Client und dem Arduino wie gewünscht? Kopieren Sie Ihren finalen Arduino-Code in das Protokoll. Gerne können Sie zusätzlich ein Foto Ihrer Testumgebung sowie einen Screenshot des Serial Monitors (mit den empfangenen Zeichen) einfügen.

*Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (auf Moodle).*

### 3.c) Java-GUI zur Steuerung der LED-Matrix

In dieser erweiterten Aufgabe kombinieren Sie mehrere Techniken und steuern die integrierte 12×8-LED-Matrix des Arduino UNO R4 WiFi vollständig drahtlos über WLAN, mit einer selbst erstellten grafischen Java-Anwendung. Ziel ist es, den aktuellen LED-Zustand aus der Benutzeroberfläche auszulesen, als kompakten Datenstring zu kodieren und an den Arduino zu senden. Die Kodierung erfolgt in Form von drei 32-Bit-Hexwerten, die den Zustand der 96 LEDs (8 Zeilen × 12 Spalten) vollständig repräsentieren.



Zunächst verwenden Sie für das Arduino-Setup das bereitgestellte Programm `aufgabe_3c_arduino_vorlage.ino`. Der Arduino spannt ein eigenes WLAN auf und wartet über Port 8888 auf eingehende Daten. Sobald ein gültiges Datenpaket empfangen wird – bestehend aus drei durch Leerzeichen getrennten 8-stelligen Hex-Zahlen – wird dieses Paket direkt an die LED-Matrix übergeben und angezeigt. An diesem Arduino-Programm müssen keine Anpassungen mehr vorgenommen werden.

Auf Java-Seite steht Ihnen das Projekt `aufgabe_3c_java_vorlage` zur Verfügung. Die Benutzeroberfläche besteht aus einem 12×8-Raster an Schaltflächen, wobei jede Schaltfläche genau einer LED auf der Matrix entspricht. Ein Klick auf eine Schaltfläche aktiviert oder deaktiviert die entsprechende LED. Der aktuelle Zustand wird direkt in der Oberfläche visualisiert: Eine aktive LED wird durch ein „X“ dargestellt, eine inaktive bleibt leer.

Nach jeder Änderung wird der vollständige Matrix-Zustand intern als binäre Zeichenkette aufgebaut (96 Bits = 12 Spalten × 8 Zeilen). Diese Bitfolge wird anschließend in 24 Hex-Zeichen umgewandelt (4 Bits = 1 Hex-Zeichen). Schließlich wird dieser Hex-String in drei Teile zu je 8 Zeichen (3 × 32 Bit) zerlegt und als Datenpaket an den Arduino übertragen.

Ihre konkrete Aufgabe besteht nun darin, in der Methode `sendMatrixState()` die noch fehlenden Teile des Programms zu implementieren. Die grundlegende Struktur ist bereits vorbereitet, allerdings fehlen die zentralen Verarbeitungsschritte. Die entsprechenden Stellen sind im Code als `TODOs` markiert. Ihre Aufgabe ist es, den aktuellen LED-Zustand korrekt in eine 96-Bit-Folge zu überführen, diese in Hex umzuwandeln und die drei resultierenden Datenblöcke über die TCP-Verbindung an den Arduino zu senden. Solange Ihre Implementierung noch unvollständig ist, funktioniert der Programmablauf dennoch mit Platzhalterwerten – so wird stets ein mittlerer Block mit aktiven LEDs und zwei inaktive Ränder dargestellt.

Bitte bearbeiten Sie alle `TODOs` schrittweise und testen Sie Ihr Programm anschließend mit einer aktiven WLAN-Verbindung zum Arduino.

**Frage 15:** Kodierung verstehen. Beschreiben Sie in eigenen Worten, wie der Zustand der 12×8-LED-Matrix intern gespeichert und für die Übertragung auf drei 32-Bit-Zahlen aufgeteilt wird. Warum ist diese Art der Kodierung sinnvoll?

**Frage 16:** Anwendung testen und reflektieren. Testen Sie Ihre Lösung, indem Sie ein bestimmtes Muster (z. B. ein „H“, ein Rahmen oder eine diagonale Linie) auf der LED-Matrix darstellen. Fügen Sie einen Screenshot der Benutzeroberfläche und einen Screenshot des Serial Monitors bei. Was würden Sie an der Benutzeroberfläche oder Datenübertragung verbessern?

**BITTE BEACHTEN: Am Ende der Praktikumsstunde sind...**

- ...die aufgebauten Schaltungen wieder abzubauen
- ...sämtliche Bauteile wieder sauber und vollständig in die Schachtel zu verpacken; Widerstände wieder in die richtigen Fächer zu geben, Drehregler/LEDs in die Fächer zu legen, etc.